

The $O(T^2)$ Bottleneck.

Self-attention compares every token with every other token, so its score matrix is $T \times T$. That single fact makes both *compute* and *memory* grow with the square of the sequence length. Below we derive the cost from scratch, tabulate exactly how fast it explodes, and preview the three escape routes. Every number on these pages is computed in full — nothing is summarised or deferred.

THE EQUATION

$$S = \frac{QK^\top}{\sqrt{d}} \in \mathbb{R}^{T \times T} \quad \Rightarrow \quad \underbrace{O(T^2 d)}_{\text{compute}}, \underbrace{O(T^2)}_{\text{memory}}$$

Worked example. A 5×5 score matrix shown as a heatmap, plus a memory table for $T = 512, \dots, 131072$ at fp16, per head and for 32 heads.

Topic 1 of 2 in this module · companion to the course page · v1.0

Contents

1	The claim: attention is $O(T^2)$	2
2	The derivation: counting the cost	3
3	Why this dominates at long T	3
4	The score matrix, made concrete	4
5	The memory table: how fast it explodes	4
6	Most of those entries are wasted	5
7	Three escape routes	5
A	Reproducible (NumPy)	5

1 The claim: attention is $O(T^2)$

Scaled dot-product attention over a sequence of T tokens, with model/head dimension d , is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \quad Q, K, V \in \mathbb{R}^{T \times d} \quad (1)$$

The intermediate object that makes attention powerful — and expensive — is the *score matrix* $S = QK^\top / \sqrt{d}$. Because Q has T rows and $K^\top$ has T columns, S is $T \times T$: one entry S_{ij} for *every ordered pair* of tokens.

Symbol	Shape	Meaning
$Q = XW_Q$	$T \times d$	queries (one row per token)
$K = XW_K$	$T \times d$	keys
$V = XW_V$	$T \times d$	values
$S = QK^\top / \sqrt{d}$	$T \times T$	raw scores — the costly object
$P = \text{softmax}(S)$ (row-wise)	$T \times T$	attention weights, rows sum to 1
$O = PV$	$T \times d$	output (one row per token)

Attention is an all-pairs operation: token i must look at *every* token j to decide how much to attend. “Every-to-every” over T tokens is T^2 comparisons — the same reason a handshake between T people takes $\sim T^2/2$ handshakes. Double the room and you quadruple the work.

2 The derivation: counting the cost

2.1 Compute

The score matrix is a matrix product $QK^\top$ with $Q \in \mathbb{R}^{T \times d}$ and $K^\top \in \mathbb{R}^{d \times T}$. Each entry is one dot product of length d :

$$S_{ij} = \frac{1}{\sqrt{d}} \sum_{k=1}^d Q_{ik} K_{jk} \quad \implies \quad d \text{ multiply-adds per entry.}$$

There are $T \times T = T^2$ entries, so forming S costs

$$\underbrace{T^2}_{\text{entries}} \times \underbrace{d}_{\text{per entry}} = T^2 d = O(T^2 d) \text{ operations.}$$

The row-wise softmax $P = \text{softmax}(S)$ touches every entry once: $O(T^2)$. The output $O = PV$ multiplies $P \in \mathbb{R}^{T \times T}$ by $V \in \mathbb{R}^{T \times d}$, again $T^2 d$ work. Adding the three stages, the dominant term is

$$\boxed{\text{Attention compute} = O(T^2 d)} \tag{2}$$

2.2 Memory

The score matrix and the weight matrix are each T^2 numbers that must be *stored* (written to memory, read back for softmax, read again for PV). At fp16 (2 bytes/number), one head needs

$$\text{bytes (per head)} = 2T^2, \quad \text{bytes (} H \text{ heads)} = 2T^2 H. \tag{3}$$

This $O(T^2)$ *activation* memory — not the parameters — is what kills long context.

The $\sqrt{d}$ scaling and the softmax are cheap, $O(T^2)$ and $O(T^2)$ respectively. The quadratic blow-up comes from one structural fact: there are T^2 pairs. You cannot change that with a better constant — only by *not* computing or *not* storing all T^2 entries.

3 Why this dominates at long T

A transformer layer also contains a position-wise feed-forward network (FFN), typically $d \rightarrow 4d \rightarrow d$. Its cost is two matrix products applied to T rows independently:

$$\text{FFN compute} = O(T d^2) \quad (\text{linear in } T).$$

Compare the two terms per layer:

$$\frac{\text{attention}}{\text{FFN}} = \frac{T^2 d}{8 T d^2} = \frac{T}{8d}.$$

The FFN is linear in T ; attention is quadratic. So a crossover exists at $T \approx 8d$: below it the FFN dominates, above it attention runs away. With $d = 1024$:

T	Attention $\sim T^2 d$	FFN $\sim 8 T d^2$	attn/FFN
512	2.68×10^8	4.29×10^9	0.06
2,048	4.29×10^9	1.72×10^{10}	0.25
8,192	6.87×10^{10}	6.87×10^{10}	1.00
32,768	1.10×10^{12}	2.75×10^{11}	4.00
131,072	1.76×10^{13}	1.10×10^{12}	16.0

At $T = 131,072$ attention does $16\times$ the FFN's work and the gap keeps growing.

4 The score matrix, made concrete

With $T = 5$, $d = 4$ and small random Q, K , the scores $S = QK^\top/\sqrt{4}$ are

$$S = \begin{bmatrix} 1.769 & -0.177 & 1.194 & -1.687 & -1.398 \\ -2.428 & 2.023 & -1.351 & 1.229 & 0.763 \\ 0.837 & -0.137 & 0.552 & 0.026 & -0.704 \\ -1.024 & 1.885 & -0.365 & 0.235 & -0.469 \\ -1.143 & 1.596 & -0.655 & -0.195 & 0.249 \end{bmatrix}.$$

After a row-wise softmax we get the weight matrix $P = \text{softmax}(S)$ — the heatmap below. Each row sums to 1; the darkest cell is where a query concentrates. The whole $T \times T$ grid is what must be stored.

	k_1	k_2	k_3	k_4	k_5
q_1	0.562	0.080	0.316	0.018	0.024
q_2	0.007	0.561	0.019	0.254	0.159
q_3	0.359	0.135	0.270	0.159	0.077
q_4	0.038	0.691	0.073	0.133	0.066
q_5	0.040	0.626	0.066	0.105	0.163

Five tokens already need 25 entries. The next section shows what happens when T grows.

5 The memory table: how fast it explodes

Using Eq. (3) with fp16 (2 bytes) per score, here is the exact activation footprint of the score/weight matrices — per head and for a 32-head model:

T	T^2 entries	Memory / head	Memory, 32 heads
512	262,144	512 KB	16 MB
2,048	4,194,304	8 MB	256 MB
8,192	67,108,864	128 MB	4 GB
32,768	1,073,741,824	2 GB	64 GB
131,072	17,179,869,184	32 GB	1 TB

Read down the table: every time T doubles, T^2 quadruples, so memory and compute jump $4\times$. From $T = 8\text{k}$ to $T = 32\text{k}$ (a $4\times$ length increase) the 32-head footprint goes $4\text{ GB} \rightarrow 64\text{ GB}$, a $16\times$ jump — a single 80 GB GPU can no longer even hold the scores. This $4\times$ -per-doubling is the whole story.

It is tempting to blame model *parameters* for out-of-memory errors at long context. They are not the culprit: parameters are fixed, but the score *activations* scale as T^2 . At $T = 131,072$ the scores alone are 1 TB — dwarfing the weights of even a large model. The fix must attack the T^2 *activation*, not the parameter count.

6 Most of those entries are wasted

Empirically, trained attention is highly peaked: a query concentrates its mass on a handful of keys and assigns near-zero weight to the rest. In our 5×5 example above, rows already spike on one or two columns. For a more realistic peaked row of length $T = 10,000$, roughly **98%** of the weights fall below the uniform level $1/T$ — they contribute almost nothing to the output yet still cost a full slot in the T^2 matrix. We pay quadratic price for a result that is effectively sparse, which is exactly the opening that the next modules exploit.

7 Three escape routes

There is no way to make exact, dense attention sub-quadratic in *compute*, but each factor of the cost can be attacked differently:

Route	Idea	Where
Sparse patterns	compute only a chosen subset of pairs ($O(T\sqrt{T})$, $O(T \log T)$)	Module 5.2
Low-rank / kernels	approximate $\text{softmax}(QK^\top)$ without forming it ($O(T)$)	Module 5.3
IO-aware exact	keep $O(T^2d)$ compute but $O(T)$ <i>memory</i> (FlashAttention)	Topic 2 (next)

The next topic takes the third route: **FlashAttention** computes the *exact* same output but never materialises the $T \times T$ matrix, collapsing the memory term from $O(T^2)$ to $O(T)$ through tiling and the online softmax.

A Reproducible (NumPy)

Inputs: `seed=7`, $T = 5$, $d = 4$; the memory table uses 2 bytes/entry and 32 heads.

```

import numpy as np
# --- score matrix + heatmap weights (T=5, d=4) ---
np.random.seed(7)
T, d = 5, 4
Q = np.round(np.random.randn(T, d), 2)
K = np.round(np.random.randn(T, d), 2)
S = Q @ K.T / np.sqrt(d)
P = np.exp(S - S.max(1, keepdims=True)); P /= P.sum(1, keepdims=True)
print(np.round(S, 3)); print(np.round(P, 3))    # matches body
# --- memory table ---
for T in [512, 2048, 8192, 32768, 131072]:
    entries = T*T
    print(T, entries, 2*entries, 2*entries*32) # entries, bytes/head, bytes/32 heads
# 512 262144 524288 16777216 ... 131072 17179869184 34359738368 1099511627776
# --- FFN crossover (d=1024) ---
d = 1024
for T in [512, 2048, 8192, 32768, 131072]:
    print(T, T*T*d, 8*T*d*d, (T*T*d)/(8*T*d*d)) # attn, ffn, ratio = T/(8d)
# crossover (ratio 1.0) at T = 8192 = 8*d

```